

摘要:

英伟达芯片因为原有设计和庞大的公共缓存, 导致处理少量数据时特别卡。OpenAI的新芯片决定推倒重来: 直接砍掉多余的公共缓存, 让计算核心和内存“一对一”直连, 并换用更灵活的计算方式。它从根本上消除了卡顿, 而不是靠堆算力掩盖。更牛的是, 他们让AI来主导芯片和软件设计, 仅9个月就造出实物, 效率彻底碾压对手。

TL;DR

假如我重生成某基模型团队的 Infra 负责人, 那么一定会对 Nvidia 的 GPU 及整个系统有各种不满, 然后自己做芯片...

对于 Nvidia 网络部分就不用谈了, RoCE各种缺陷, 然后 DPU 也有一堆性能和安全问题, 也难怪今年 Nvidia 又开始吹 Scale-in 了, 但是一个从来没做过云的团队这方面想成熟起码还要 5 年. 举个例子, 最近我们做了一件事情, 在我们 MaaS 在线业务中的某个 PD 分离的场景中, 将传输协议用了 CIPU 的 eRDMA 替换直接把 TTFT 降低了 40%, 直接等同于维持 TTFT SLA的情况下可以省大量的 Prefill 计算服务器...

对于 GPU 微架构, 去年就对此详细写了一篇《[Inside Nvidia GPU: 谈谈Blackwell的不足并预测一下Rubin的微架构](https://github.com/zartbot/blog/issues/3)》(英文版: <https://github.com/zartbot/blog/issues/3>) 批评 Nvidia Blackwell微架构的各种缺陷, 老黄还在 NV 内部转发过...

当时建议 SM 增加用于调度的超标量核, 这不 OpenAI Jalapeño 就有了. 另一个问题是我一直给 Nvidia 在提, 你们 GPU 架构中, 大量的 L2 Cache 占用了太多芯片面积, 但是似乎对它又很难去做一些控制, 缓存层次结构比较厚, 为了维持 UMA 实质上在 L2 两个 Partition 上会增加 200~400 cycle 的延迟, 而且对外 I/O 的交互也无法直接写到 SMEM 内, 这下好了 OpenAI Jalapeño 直接砍掉 L2 Cache. 还有前几个月在卷 Anthropic 的一个面试题, 实际上也是在准备和自研训练和推理芯片相关的任务, 并且在内网也详细分析了各种加速器架构以及该怎么做...

今天这篇分析 Jalapeño 的文章, 我假设我自己是某基模芯片团队的负责人, 从模型的需求以及当前 Nvidia GPU 的缺陷出来, 逐步从第一性原理开始推导该如何设计, 当然也会在公司内部逐渐复现整个设计过程. 通过这种方式来阐述 Jalapeño 的芯片架构以及背后的一些工具链... 本文目录如下:

1. 从 Nvidia GPU 谈起
 - 1.1 推理场景下的理论峰值
 - 1.2 架构决定 E2E 延迟
 - 1.3 Nvidia GPU 的缺陷
2. 如何设计推理芯片
 - 2.1 推理芯片第一性原理
 - 2.2 推理 Infra 现状
 - 2.2.1 不同阶段的性能需求
 - 2.2.2 KVCache 的代价
 - 2.3 推理芯片体系结构
 - 2.3.1 编程接口的变化
 - 2.3.2 体系结构的取舍
3. OpenAI Jalapeño 芯片架构
 - 3.1 芯片架构 Overview
 - 3.1.1 Core Slice 架构
 - 3.1.2 NOC 架构
 - 3.2 Core Slice 架构详解
 - 3.3 内存子系统
 - 3.4 片上网络
 - 3.6 系统架构
4. 软件架构
5. 芯片设计
 - 5.1 关键前提: 先换语言, 再让 AI 上场
 - 5.2 设计如何闭环
6. 未来展望
 - 6.1 对比 Nvidia GPGPU 架构
 - 6.2 OpenAI roadmap
 - 6.3 一些分析总结

⚠ 很多更详细的评估以及复现整个设计流程的工作就不公开了... 由于微信公众号排版不适合电脑前阅读, 因此在 Github 上放了一个重新排版的

中文版: <https://zartbot.github.io/blog/arch/jalapeno/index.html>

1. 从 Nvidia GPU 谈起

首先想起的是一个段子, 也是现状.

产品管理的兄弟: 隔壁NV做了, 为啥咱不做?

也是产品管理的兄弟: 隔壁NV都没做, 为啥我们能做?

还是产品管理的兄弟: 隔壁NV做了, 为啥我们还要做?

依旧是产品管理的兄弟: 隔壁NV做了, 我们怎么敢做?

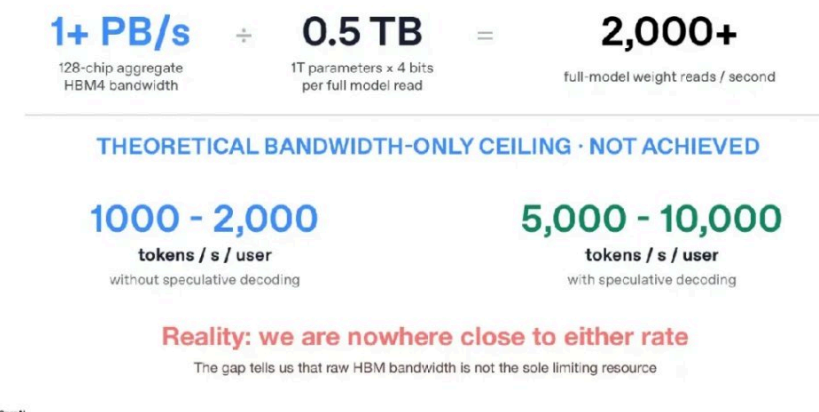
大多数做加速器的厂商, 满脑子的是抄 NV 作业, 然后美其名曰 CUDA 生态兼容, 但再怎么抄整体落后 NV 两到三代... 另外一个极端就是总想搞一些奇奇怪怪的体系结构, 试图弯道超车.... 然而几乎所有的芯片厂商和 NVidia 最大的差距在于全栈的能力, 从模型结构和算法, 再到软件生态, 最后到芯片实现, 其中哪些地方可以有 trade-off, 每次交易赢了什么输了什么, 很少有人能够讲清楚. 恰好我从算法到芯片全栈都懂, 因此可以完整的来阐述整个芯片设计的过程以及 Nvidia GPU 的缺陷.

1.1 推理场景下的理论峰值

GPU一直以来“靠大量并发 warp 隐藏访存延迟”的机制处理大量的数据, 这个机制本质上依赖大 batch. Batch 小了, 可调度的 warp 就少, 流水线填不满, 于是远离 roofline. 这便是推理场景中经常遇到的问题, 延迟隐藏机制失效.

既然推理阶段很大程度上是 Memory bound 的, OpenAI 做了一个很简单的计算, 假设累计 ScaleUP 域有 N 个芯片, 每个芯片 HBM 带宽为 M, 估计一个模型参数规模为 K, 那么可以简单的假设计算没有任何瓶颈时, 理论上输出 tokens 的最大速度为

Raw HBM bandwidth implies extraordinary token rates



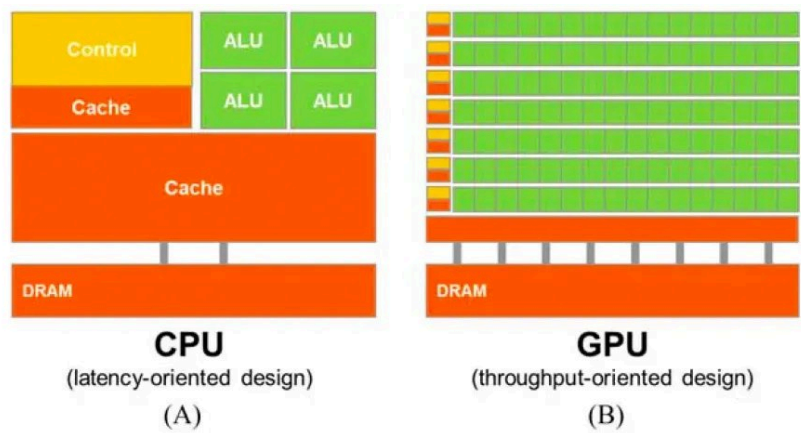
也就是说理论上, 不加投机解码, 1T 参数的模型应该能过做到 1000~2000

tokens/s/user. 但现阶段大多数推理框架在B300一类的平台上, 做多只能 100~200

tokens/s/user, 只有 10% 的理论性能. 因此这一页在说: 就算把 HBM 带宽用到理论极限, 得到的延迟仍然有一个下界. 也就是说, 单靠加带宽解决不了低延迟问题. 既然带宽不是唯一变量, 那么一定是体系结构上的问题了...

1.2 架构决定E2E延迟

在一年多前一篇文章里我暗示过, 从处理器的 workload 来看, 在现代计算机体系结构中, CPU的核心设计哲学与GPU截然不同: GPU通过海量并行线程掩盖延迟以最大化吞吐量, 而CPU则是不惜耗费巨大的芯片面积与晶体管功耗预算, 穷尽一切微架构手段将单线程执行延迟 (Single-Thread Latency) 压缩到极致.



CPU通过分支预测提前探路、多发射与乱序执行见缝插针地并行计算, 依托多级缓存拉近数据距离, 并利用硬件缓存一致性实现多核间纳秒级的就地数据共享与免内存同步, 从而掩盖了单核计算、主存访问及多核协作的等待延迟.

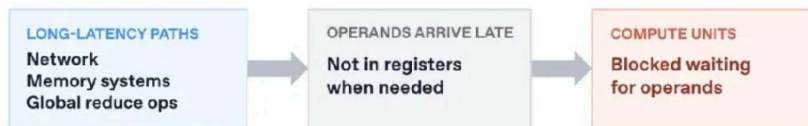
与之相对, GPU将绝大部分晶体管预算倾斜给密集的算术逻辑单元, 通过SIMT架构组织起数以万计的并发线程, 依托硬件级超快线程上下文切换在某些线程等待访存时瞬间调度其他就绪线程, 但是它根本不在乎单条指令的快慢, 而是用绝对的并行度将数据流的整体处理延迟彻底淹没在海量的计算任务中.

而对于 LLM 推理来看, 它刚好落在下表中 A* 这个位置. batch=1 时一层就是几个 GEMV, 同时还要在毫秒量级穿过数十层, 还要在每层中间做完大量的并行计算后的结果同步.

	延迟敏感	延迟不敏感
数据规模大	A*	GPU
数据规模小	CPU	NA

同时 OpenAI 在 HotChips 上的 Session 有一页专门阐述了这个问题: 我们实际测量到的延迟, 很大程度上是由底层架构本身决定的.

Architecture can dominate observed latency



What causes long latencies and late operands?

Unified memory subsystems tend to highly contended paths
Independent cores out of sync make global memory fences expensive
Centralized resources mediating network access

→ NOT hardware speed of light

Utilization falls when compute/memory blocked waiting on data in transit

OpenAI

图中的 长延迟路径 → 操作数迟到 → 计算单元阻塞 为传统的 roofline model 补上了另一个维度, 对于 GPU 芯片而言并不是简单的 Compute bound 和 memory bound 的 workload 区分, 而是还要加上一个 阻塞, 此时没有任何资源被打满, 但所有资源都在等. 利用率掉下去, 而两条天花板都还很远.

1.3 Nvidia GPU 的缺陷

我们在这一节详细展开阐述一下 OAI 那一页的观点.

首先我们来谈 OAI 提到的 Unified memory subsystems tend to highly contended paths, 它采用统一 L2 + 全局编址, 任何核等价访问任何地址. 但是这样的统一访问虽然编程上隐藏了硬件的复杂性, 但是也带来了大量的工程约束. 例如 SM 太多带宽需求太大, 从 Ampere 开始 L2 拆成了 2 个 Partition, 在 Hopper 上 cross partition 会增加 200 cycles 的延迟. 而在 Blackwell / Rubin 上还会带来更多的影响, 每个 Die 有两个 L2 partition, Dual-Die 会有 4 个 Partition. 在 Blackwell 上会增加接近 400 Cycles 的延迟, Rubin 还需要考虑到 HBM4 的累计带宽已经快接近 L2 带宽了, 同时还要受到 NV HBI 的影响, 我个人认为 Unified Memory Access 将会导致 HBM 22TB/s 的实际带宽利用率只有 60%~70%. 并且当并发请求超出端口服务能力时, 产生的排队延迟 (Queuing Delay) 其开销远超数据传输本身.

另一个问题是在一些数据同步和 warp 调度上, 在现代的 GPU 架构中, 每个计算核有了独立 PC、自由异步推进, 并且配合 TMA 这些异步内存访问器件增加了吞吐, 但正因为计算核是独立的, 硬件就无法预知谁在什么时候到达, 于是一次全局 fence 必须真的去“问一圈”, 甚至在一些算子上需要整个 system level cross ScaleUP/ScaleOut network 进行 barrier. 这就是 OAI 谈到的 Independent cores out of sync make global memory fences expensive.

最后 Centralized resources mediating network access 讲的是当成百上千个核心需要向外部网络发起通信时, 若通过集中式的 DMA 控制器、MMIO 接口或共享网卡端口进行收发, 仲裁逻辑 (Arbiter) 会成为严重的吞吐瓶颈与延迟热点, 从而将硬件本身的极速传输能力彻底拖垮. 这个涉及到一系列复杂的互联系统的问题, NOC / ScaleUP / ScaleOut 都有各种问题, 例如 KVCACHE 拷贝时采用的 CopyEngine 或者 ScaleOut NIC 上 doorbell 的约束, 以及很多通知无法支持直接写到 SMEM, 因此只能写到一些 Remote GMEM, 然后 CUDA Core polling GMEM.

正是因为这些原因, 我过去几年才会对 Nvidia 提出 SM 内需要一个超标量核让编程者能够更好的控制指令发射和调度, 同时在 L2 上给用户更多的控制能力, 进一步和 SM 内的 SMEM 融合... 而恰好 OpenAI Jalapeño 也作出了这样的选择, 下一章我们将从推理业务第一性原理的角度逐渐展开, 如何设计一颗适合推理的芯片.

2. 如何设计推理芯片

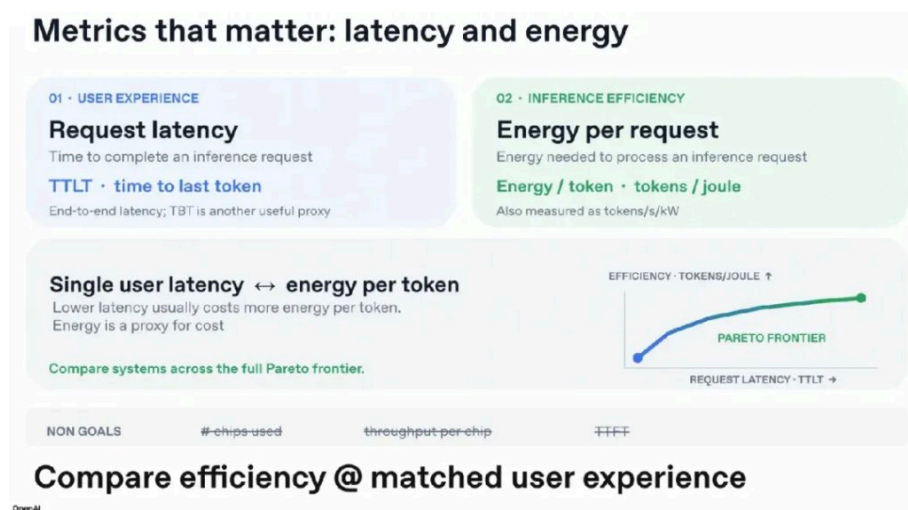
我们仔细分析发现前一节所讲的缺陷, 它们全都是通用性的代价, 需要强调, 这并没有在说 GPGPU 设计得差, 因为每一条决定当初都是对的, 它是 Nvidia 成功的关键, CUDA SIMT 的编程范式在过去十多年极大的简化了大家编写并行计算程序的复杂度. 但是这个前提在 LLM 推理场景带来了大量的问题, 然后伴随着 Coding Agent 的发展, 各种 Auto-Kernel(eg. KDA)百花齐放, 似乎我们也不需要为这些通用性付出高昂的代价了...

但这些仅仅是从芯片体系结构的视角, 更重要的是, 我们需要考虑从整个推理业务视角作为第一性原理.

2.1 推理芯片第一性原理

从第一性原理来看, 以一个推理服务经营的视角主要就是基础设施成本和供应链约束下, 满足用户体验的SLO下尽可能多的输出 tokens. 而供应链在北美最大的约束可能主要是在电力供应上.

因此 OpenAI 的演讲稿故意不把chips used,throughput per chip和TTFT列为主目标. 它选择的是 request-level user experience 和 energy per request.



吞吐能够通过并发掩盖某些等待, 单个任务的依赖链却不能凭空并行. 因而使用TTLT, 即time to last token, 表示整个请求完成时间, 同时用TBT, 即 time between tokens, 近似流式交互速度. 但我们注意到, 降低 batch 通常会改善TBT, 却让权重和网络复用下降, 每token能耗上升. 提高 batch 则通常提高tokens/s/kW, 但请求排队和每用户token间隔变长. 因此存在一个响应延迟 (TTLT) vs 能效 (tokens/Joule) 的 Pareto 前沿. 形式化的描述为:

$$\text{maximize } \frac{\text{requests}}{\text{second} \cdot \text{watt}}, \quad \text{s.t. } t_{E2E} \leq \text{SLA}$$

而 LLM 推理, 实际上是 CPU 界研究了三十年的单线程延迟问题...

CPU 界的答案	LLM 推理里的对应物
分支预测: 猜下一步, 错了回滚	投机解码: 猜后 γ 个 token, 错了丢弃
乱序执行: 在依赖链的空隙里找独立工作	乱序核
缓存 + 预取: 把延迟从关键路径上挪走	硬件管理 L1 + 依赖良好的预取, 而非 scratchpad + DMA
SMT: 用另一个线程填充空隙	用另一条序列填充空隙 (请求级批处理)
加宽发射宽度收益递减 (ILP 上限)	加投机深度收益递减 (接受率上限)

那么从体系结构的视角来看得出一个很简单的结论:

尽量提高 Data Locality, 降低数据搬运的成本.

从E2E延迟的视角, 进一步提高指令执行的并行性, 降低空闲等待的时间.

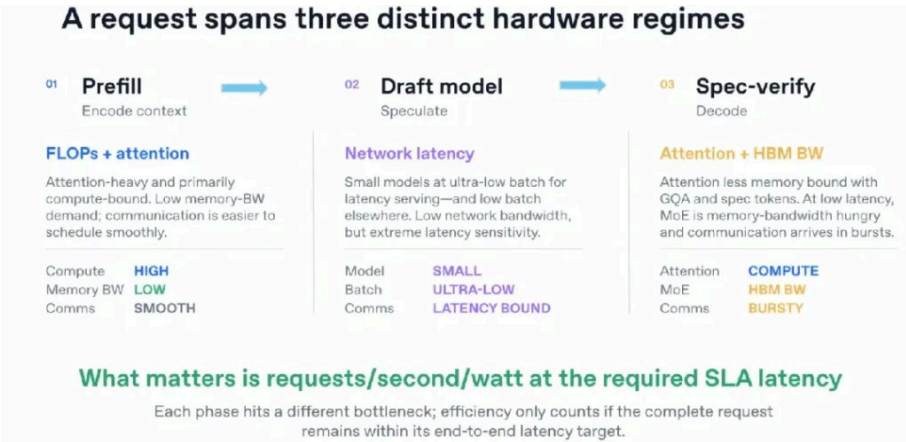
接下来几个小节我们将从推理的技术细节详细展开分析...

2.2 推理 Infra 现状

2.2.1 不同阶段的性能需求

通常对于一个推理系统来看, 主要业务分为Prefill, Decode, SpecDecode 三块, 可能我们原来的定性的结论是 Prefill是 Compute Bound, Decode是 Memory Bound, 但实际上我们从E2E 延迟的视角还需要考虑 SpecDecode 时的 Draft 模型和 Verify 两个阶段, 它们都会处在roofline 的不同位置.

OpenAI在 hotchips 的session 上也谈到这个问题, 但是其表述是不完善的. 例如在第三格, 主标题是Spec-verify, 副标题是Decode,但的自回归decode与的批量verify处在roofline上两个不同位置, 相差倍算术强度.



因此我们严格的把它分为四块, 为了简化所分析的问题, 此时我们以单个 Request 的处理进行分析

1. Prefill: Prefill一次处理完整输入上下文. 对 hidden-dim 为, 长度为 的普通Transformer模型, projection 与 FFN 的主计算量近似随 增长, full attention 的 score 部分随 增长. 大 带来足够的 M 维度, 权重可以在许多 token 之间复用, 所以 tensor engine 容易进入高利用率区. 因此它是一个 Compute-bound 的计算, 即 compute high, memory BW low, 另外对于通信而言, 它的 collective 消息随 增长, 属于带宽受限而非延迟受限, 因而容易平滑调度.

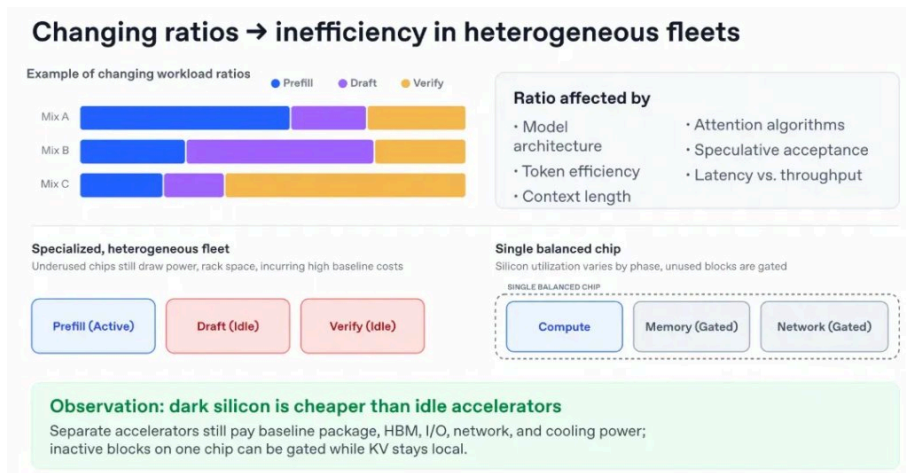
3. Draft Model: 通常和模型相关。经典自回归草稿模型和EAGLE类方法需要执行多次串行起草步，因而经常受权重/KVCache带宽和同步延迟限制。DFlash使用一次块并行扩散主干网络前向传播同时生成整个草稿块。DSpark使用一次并行主干网络前向传播，再叠加一个低成本Markov或RNN串行头。对DFlash/DSpark而言，草稿权重只需按主干网络调用读取，算术强度可随提升，瓶颈可能转向矩阵算力，目标隐藏特征/KV注入带宽，vocabulary projection 或轻量串行采样延迟。

总结成一个表如下:



如果我们采用专用的芯片应对每个阶段, 对于资源的配比也会出现问题. OpenAI 也谈了这个问题:





OAI阐述的是当前各种 PD 分离和 AFD/SpecDecode 这样的 LPU 部署带来的缺陷. 原页面沿用它自己的三分法, 所以只画了prefill, draft, verify三个比例. 实际上按照我们前面所分的4个阶段分别放入4种专用设备, 看似能让每种芯片都针对自己的kernel做到最优. 问题在于设备台数在部署后是离散且相对固定的. 设4个池的能力为, 到达工作量为, 下标依次对应prefill, decode, speculate与verify, 则可服务速率受最紧张的那个池限制:

例如我们来分析 SpecDecode 接受率动态变化的影响, 与此消彼长, 而拨动它们的不是请求. 一个请求的输出token只有两条出路, 走纯decode, 或者走 Draft + Verify. 记输出token中经 SpecDecode 路径提交的比例为, 为每秒已提交的输出token数, 则

其中是一轮 Draft Model 的成本, 自回归为步而块并行为1次宽pass, 则正比于该轮的物理 token 预算. 不由请求混合决定: 塌陷把它压向0, 高并发下调度器按前缀存活概率裁掉低置信后缀也把它压向0, 而接受率回升时它又回到1.

于是在同一批请求, 同一个模型, 同一个context length下, decode池与verify池的负载可以走向两个相反的极端. **decode池必须按配齐, draft与verify池必须按配齐, 而这两个峰值不会同时到来:** 资源预留的是峰值之和, 用的是其中一组.

四个阶段的限制项落在不同资源轴上: prefill是矩阵算力, decode是HBM权重带宽加逐层同步延迟, 自回归起草是draft权重带宽加次同步, verify是去重expert的权重流量加EP all-to-all突发. 所以这四种专用芯片不是同一款芯片的大小号, 而是算力, 带宽与网络配比互不相同的四种芯片. 接受率上升时, verify的次数下降; context length上升时, prefill与KV的成本提高; 模型换成expert占比高的MoE时, verify的网络与HBM压力又增加. 一个pool成为瓶颈时, 另外两个pool仍要支付 package, HBM, I/O, 网络与散热的基线成本. 由此得到一个容易漏掉的后果: 即使运维愿意把设备从闲置的池搬进 瓶颈池, 搬过去也无法使用, 因为prefill芯片的HBM与算力之比本来就不是按 decode配的. **固定的设备台数比例等于同时锁死五个资源维度的比例.**

2.2.2 KVCache 的代价

在传统的推理部署中, 我们通常会考虑 PD 分离的部署方式, 而此时大量的 KVCache 的搬运也带来了极大的功耗以及网络同步的损失. OpenAI 在 hotchips 中的阐述如下:

这一页把架构选择压成一个二选一: 异构要么发生在系统之间, 代价是 KV 跟着不同的阶段走, 这就是现在 PD 分离的做法, 但是每个阶段的网络和同步所带来的影响以及KVCache 搬运的功耗是巨大的.另一种选择是将不同阶段的 KVCache 移动放在芯片之内, 代价是活跃资源的比例跟着不同的阶段变化.

通常 KVCache 的大小为 和序列长度成正比, 即便是一些 Linear Attention 的算法也需要在多次交互中为每一轮留下 Snapshot. 数据的搬运会导致 ScaleUP/ScaleOut 以及 NOC 对原有的集合通信的性能影响, 同时还要考虑大量的 KVCache 从 Prefill 节点的 HBM 读出, 再从 Decode 节点 HBM写入, 依然会影响到原有的推理算子的执行. 具体的分析可以从时间比, 每轮边界, HBM容量与事务语义 四个方向展开进行量化的分析.

由于涉密这部分就不公开多说了...还记得本文开头所讲我们 MaaS 服务将传输协议换成 CIPU eRDMA 后, 推理引擎未做任何代码修改 TTFT 下降了 40%...

最后对于芯片的设计, 如果用几种独立加速器, KV cache 需要在设备之间移动. Jalapeño 的策略是把异构能力放在一颗平衡芯片内部:

当前阶段需要 tensor compute, 就激活更多计算单元.

当前阶段受 HBM 限制, 就强调本地内存路径.

当前阶段通信密集, 就使用 collective 和 scale-up 网络.

暂时不需要的模块进行 power gating.

KV cache 尽可能留在本地.

2.3 推理芯片体系结构

几个月前 David Patterson 有一篇文章《Challenges and Research Directions for Large Language Model Inference Hardware》^[1] 在阐述推理硬件的体系结构, 对于内存子系统, 使用 HBM / 3D-DRAM / HBF 有很多讨论, 但是为了简化问题, 例如新的 HBM 可以构建 PNM 和 Customized Based Die 将 Memory Controller 和 HBM PHY 从计算 Die 中移出获得更多的算力资源, 或者像 AMD 那样使用 XCD 和 FCD 的 3D 封装, 可以进一步提高 XCD 的良率以及更大的 L2 Cache 和互联NOC, 我们暂时不讨论这部分的内容, 更多的关注在计算 Die 内部的微架构以及互联架构.

具体的内容在公司内部也有一个详细的文档分析了各种算力芯片的微架构, 包括 Nvidia GPGPU / LPU / Google TPU 和华为 Ascend 950. 并且针对 Agent based compiler 这些工作进行了一系列探索.

2.3.1 编程接口的变化

前几个月我在做一个 Anthropic 公司招聘用的性能工程相关的家庭作业面试题, 任务代码在: https://github.com/anthropics/original_performance_takehome, 大概的内容是在一个自定义的 VLIW + SIMD 处理器上进行性能优化. 通过使用Claude-Opus模型进行优化, 后来刷到第二以后因为差距已经在2%以内, 以及一些计算资源和 token 费用的问题就没有继续做下去了, 仿佛现在还是第三...

通过这个题目让我意识到它就是在设计一个专用 ASIC 芯片用于推理和训练的编译系统, 似乎原有的 CUDA 生态和 SIMT 的编程接口虽然对人友好, 但带来的资源开销是巨大的, 然而有了 Agent as compiler 了以后, 并且伴随着 Ligeng's KDA 这样的工作, Kernel 优化似乎可以通过 Agent 很好的完成了, 似乎再也不需要去照顾人类的心智缺陷, 而充分的发挥硬件的能力.



那么原有的 SIMT 的抽象和 GPGPU 在推理场景下的优势也就不存在了, 通常一个模型的架构在预训练阶段就已经定死, 我们会有两个月的时间充分的对推理平台进行基于 Agent 的调优. 这也是 OpenAI Jalapeño 体系结构上解除的一道枷锁.

2.3.2 体系结构的取舍

去年的blog也讨论过一个问题, Nvidia 的 Warp 调度机制和延迟隐藏机制并不适合推理场景. 另一方面在文章《谈谈GPU的内存模型及互联网络设计》中也谈到了 Nvidia GPGPU 复杂的内存语义, 数据路径上存在 Genric Proxy 和 Async Proxy, 如果需要 ScaleOut 还要额外的同步机制, 然后整个 NOC 和内存层次结构也越来越重, 而 Warp 调度的机制来看并不能让我满意, 每一次数据搬运都需要显式编排, 而编排本身有开销, DMA 描述符的构造、启动、以及等待完成的 barrier 等等...

假设一个 tile 的计算只需要 500 个周期, 而 DMA 编排 + barrier 开销是 100 个周期, 那么固定开销占了 20%. 但是在 GPU/TPU 的典型场景 (大 batch, 一个 tile 算几千周期) 里, 同样的 100 周期只占 2%, 完全可以忽略. 很不幸的是, Decode 阶段正好落在前一种情况: batch 小, 每个 tile 的计算量小, 于是同步开销从“可忽略”变成“主导项”. 这时候把同步从“显式 barrier”换成“硬件依赖跟踪 (乱序 + cache)”就有了正收益.

因此, 我希望他们能在 SM 内加一个超标量核进一步来控制延迟, 正好这次 OpenAI Jalapeño 中的 Out-of-Order Core 正好也验证了这个判断.

同时, 前一段时间也对 Blackwell 系列进行了完整的指令 Cycle 级的测量和完整的芯片级分析, 对外公开了《Dissecting SM_120 through Microbenchmarking》^[2]. 而对于 B300 片上网络 / L2 Cache 和功耗管理上也发现了大量的问题. 简而言之就是整个内存层次结构太重了, 而片上网络中, 为了维持 L2 的带宽, 很早就被设计成了 2 个 Partition, Nvidia 期望继续维持 UMA 的编程模型来隐藏不同 L2 Partition 访问的延迟差, 但是在 Multi-Die 的 Blackwell / Rubin 上代价太大, MultiDie的结构使得它单颗 GPU 有了 4 个 L2 Partition, 再维持 UMA 在推理阶段引入的访问内存损失就多达几百个周期, 显然是得不偿失的.



另一方面是在 RDMA 这些场景, 很多通知机制我希望能够 bypass L2 直接写到 SMEM, 这样 SM 就可以很快速的 polling SMEM 获得相对确定性的延迟, 而当前 polling GMEM 很有可能会被 warp scheduler yield导致更大的延迟. 并且 L2 Cache 的机制我们是很难控制的, 很有可能我们关键的一些同步用的 counter 被 SM 大量的内存访问 evict, 我一直也在跟 Nvidia 提什么时候能够把 L2 Cache 也划分出一部分作为 SMEM 那样可以自己控制的器件....

还有进一步的一些问题, 例如由于同步机制的影响, 无论是 intra-chip 还是 inter-chip 的集合通信性能都会受到延迟的影响, 这也导致了场景下 TP 并行的规模无法进一步扩展成为显著的影响点. 而这些问题似乎都来自 UMA 的影响, 那么很自然的一个想法, 能否做成一个 NUMA 结构, 并将关键的集合通信相关的处理在 NOC 和互联(ScaleUP/Scale Out)进行充分的优化?

当然我能理解 Nvidia 作为一个商业公司要去追求 peak throughput 的原因, 但是对于推理系统而言, 我们从目标函数出发, 有两条独立的推论, 分别落在微架构和系统架构上.

推论 A: 有效 token 必须靠“低延迟下的高利用率”取得, 不能靠堆 batch, 具体来看:

- 缩短访问内存的路径: 对于内存延迟放弃隐藏的策略, 而通过更少的内存层次消除, 可以接受 NUMA 的代价
- 更高的单指令流的ILP: 使用乱序核 + 硬件 L1 + 预取的方式, 而不依赖 occupancy
- 适当的矩阵单元大小: 保证小batch时也不会有性能损失
- 消除 Kernel Launch 与同步的固定开销

推论 B: 芯片面积不再是稀缺资源, 闲置整机才是浪费.

- 拒绝异构的各种分离技术, 同构集群避免专用资源调度瓶颈
- 不以 peak FLOPS 为目标, 充分考虑推理各阶段的资源需求, 做到算力/内存带宽/通信的平衡
- 允许部分的暗硅, 保证芯片满足各阶段的需求
- 平衡整个芯片的功耗, 并以它作为芯片设计目标
- 降低通信的延迟, 特别是长尾延迟

整个在体系结构上的决策可以概括为下面这张图:

而这些体系结构方面的思考和 OpenAI 不谋而合, 具体的分析我们将在下一章展开.

3. OpenAI Jalapeño 芯片架构

Jalapeño 应该是工业界第一款 AI 加速的软硬件协同设计的芯片平台, 硬件上从第一行 RTL 到最终 tapeout 仅用了 9 个月, 当然这和整个芯片架构也紧密相关, 由于没有复杂的同步和硬件调度机制, 这使得验证的周期加快了很多.

另外为了time-to-market, 整个芯片复用相对成熟的Ethernet based Scale-UP. 整个工业界第一个基于标准 Ethernet 构建的支持内存语义的 ScaleUP 应该是我在 2020 年开发的 NetDAM, 这也是一个很简单的工程上的 trade-off, 通用交换机芯片的供应链非常好, 成本也很低, 虽然延迟和有效利用率还有一些缺陷. 但是对于推理芯片互联所需要的极致低延迟互联, 这只是一个暂时的方案, 即便是后面 BRCM 推出的 SUE 也是较大的缺陷的, 跟他们交流的过程中, 我告诉他们可以参考 FinePack 的一些方案, 但是最后发现 BRCM 并没有完整的知识背景, 没有考虑到一些非常细节的排队延迟, 特别是一些长尾延迟的抑制...

3.1 芯片架构 Overview

Jalapeño 的芯片架构如下图所示:

它放弃了 Nvidia 那样的 UMA 结构, 而是将计算核和 HBM Slice 一对一的进行配对, 构建最低延迟的内存访问路径. 然后片上网络专门针对cross-die 集合通信进行了特殊的低延迟大带宽优化. 而且还保留了一条用于兜底的基于通用以太网的 ScaleUP 互联路径.

这样的设计也是软硬件协同的结果, Jalapeño 的很多硬件简化 (删掉统一 L2、放弃跨卡内存语义、暴露 NUMA 距离) 都把复杂度推给了编译器和 kernel. 我们注意到底部的那句话, 实质上就是通过**explicit placement**, 数据落在哪个 slice 是**被明确安排的**, 并通过编译器构建了很好的优化, 后面我们将在软件系统架构中详细展开. 同时, 芯片没有中央调度器, 每个 core 独立取指与推进, 这样的好处是防止"数据移动和同步"主导执行时间.

整颗芯片采用多个 Die 封装, 如下图所示:

Compute Die 采用 TSMC N3P 工艺, 基本上顶满了 reticle-size, 整体算力 MXFP4 支持到 13.4 PFLOPS, 外接支持 6 颗 HBM4. 但是将 IO 独立到一个单独的 IO-Die, 并采用 TSMC N3E 的工艺. IO 与计算解耦, 意味着下一代可以只换 compute die 而复用 IO chiplet, 或者反过来, 同时也降低了对 Compute Die 的资源占用. 对一个刚起步、要快速迭代的项目, 这是有价值的自由度. 并且整个 ScaleUP 并没有极致的追求更大的带宽, 仅支持 600GB/s local(in-rack) 和 200GB/s global(cross-rack).

另外我们注意到它的 Matrix 引擎原生支持 MXFP8 × MXFP4, 这是 MoE 中常用的激活 FP8 × 权重 FP4 路径, 另外整个芯片应该还是有针对 Attention 计算的 BF16 支持能力.

3.1.1 Core Slice 架构

从已知的公开信息, 我们**推测**它的计算核结构如下:

首先整个程序采用一个 GigaKernel 执行, 类似于 CUDA 中的 Persistent Kernel, 但是由于 Jalapeño 更多的要考虑延迟, 因此不会有 CUDA Core 那样的多个 warps 占用的情况, 更多的是靠内部的超标量结构构建的单个指令流内部的预取与乱序窗口来隐藏延迟, 这就是它的前端取指 / 译码 / 乱序发射 的来源. 为了保证整个核心高性能的运行, 应当存在一个相对独立的同步锚点, 执行类似于GPU中的__syncthreads / mbarrier 相关的能力.

然后在计算单元, 一般都是标准的 Tensor/Matrix Unit, SIMD/Vector Unit 和 Scalar Unit 构成. 最关键的部分在于 Matrix Unit 的架构. 我们期望它能够在推理 Decode 时小 batch 时有更高的利用率, 同时整体来看, 也能满足 Prefill 这样 Compute bound 的计算. 而这里便是整个芯片最关键的一环, 它是如何配合乱序执行? 最佳的Shape是多少? 延迟如何隐藏的? 我们在稍后的一节详细展开.

接着是内存子系统, 它并没有像 Nvidia 的 SM 那样配置 RegisterFile / L1Cache / SMEM / TMEM, 而是仅有一个 L1 Cache, 同时整颗芯片也没有一个全局的 L2 Cache. 然后 HBM 直接分 Slice 接到每个 Core Slice, 这样很好的避免了在 NOC 上的 contention.

和 Nvidia 最大的区别在于, 它的 Tensor Unit 和 SIMD Unit / Scalar Unit 之间不需要将数据频繁的在 SMEM / TMEM 和 Register File 以及 L1 Cache 之间频繁的进行搬运和拷贝, Tensor Unit 和 SIMD Unit 直接通过 L1 Cache共享操作数, Tensor Unit完成计算后, 直接就可以 issue SIMD 指令继续计算. 这样消除了 barrier / DMA编排等固定的开销, 然后 小的 Shape 进行计算对于 Decode 阶段的利用率也非常友好, 同时这块 L1 也可以很好的吸收掉 HBM 带来的抖动.

3.1.2 NOC 架构

在 NOC 上, 由于 Jalapeño 是一个 NUMA 架构, 并且在推理场景中大量的并行操作需要进行集合通信, 因此 OAI 为它构建了一个超低延迟的支持集合通信的独立 NOC. 而同时它还支持一个相对低速的传统的 NOC 用于 NUMA 节点之间的 HBM 内存访问和连接 ScaleUP IO Die.

3.2 Core Slice架构详解

我们需要考虑设计的这块芯片在Prefill, Decode, Draft和verify四个阶段都能够平衡的满足业务需求而不出现明显的短板, 并且在性瓦比(Perf-Watt Ratio)上获得数倍的收益. 那么针对 Decode batch小的情况, 需要详细考虑 Tensor Unit 的设计. 从第一性原理出发, 我们可以评估一些常用的模型的矩阵形状, 最终可以给出一个结论, 满足最小整除并不需要额外 padding 的 Tensor Unit shape 为 64x64. 然后考虑到整个系统是 latency-driven 设计的, 并且已知有大量的 Out-of-Order 的指令 issue. 并且整颗芯片移出了大量的 L2 Cache占用, Global NOC的资源占用, 因此我们可以在单个 Core Slice 内构建多个 Tensor Unit, 把芯片面积更多的花在 Tensor Unit 上.

因此我们猜测它的Tensor Unit是一个 64x64 @ MXFP4 的 Output-Stationary 的结构, 并且由于延迟的需求, 没有使用脉动阵列而是使用了一个常见的加法树结构. 然后整个 Core Slice 根据 13.4PFLOP/s 以及芯片频率 1.7GHz 倒推, 单个 Tensor Core MAC 为 64x64x2, 累计 64 个 Core Slice. $13.4\text{PFLOPs} / 1.7\text{GHz} / 64 / (64 \times 64 \times 2)$, 单个 Core Slice 包含了 15 个 64x64 Tensor Unit. 由于它是一个很大的 reticle size Die 的结构, 考虑到制造良率的影响, 而且整个芯片来看没有 L2 因此我可能会考虑放置 16 个 Slice 但仅激活 15 个的做法. 然后这些数据和公开的 MXFP8 等浮点性能也可以完全对齐. 稍微补充一下, 官方并没有公布 BF16 的峰值浮点性能, 我们可以根据这个结构倒推出来峰值在 835TFLOP/s. 可以看到相对于 B300 的 BF16 2.5PFLOPs 小了很多, 但这样的结构设计的取舍是值得的, 通常在推理过程中 Attention 的计算会用到 BF16 的精度, 而 Expert-FFN 的计算通常为 FP8xFP4, 而且 Attention 的计算中还要涉及到大量的 Softmax 等 SFU 处理, 通常每个 head 的 dim 也相对较小.

整个 Core 内的结构如下:



所有的操作都是只要有空闲的 Tensor Unit, Out-of-Order Core 可以进行多次提交提高指令并行度, 而数据依赖都由 L1 Cache 状态决定, 数据也可以硬件控制做Prefetch. 对于 L1 Cache 的大小, 我们根据 HBM 的带宽和延迟, NOC 互联的带宽以及 Tensor Unit 的处理能力等几个维度进行了详细评估, 每个 Core Slice估计 L1 Cache大小为 512KB. 对比 Blackwell 芯片, 实际上 Blackwell 芯片会有 RegisterFile / L1 Cache / SMEM / TMEM 等几块存储区域, 而 Jalapeño 很干净的处理为一整块区域并连接到 HBM Slice.

这正是整个 Jalapeño 的核心决策. 用更多的芯片面积给多个 Tensor Unit, 剔除复杂的 Global L2 / 调度器 以及 SM 内部的多种存储结构. 然后也无需追求更高的峰值 FLOPs 而是更加关注实际

工作中的 可维持的FLOPs. 然后在一些 workload 较轻时, 还可以通过 power gating 关闭掉一些执行单元, 获得更高的 Perf-Watt Ratio.

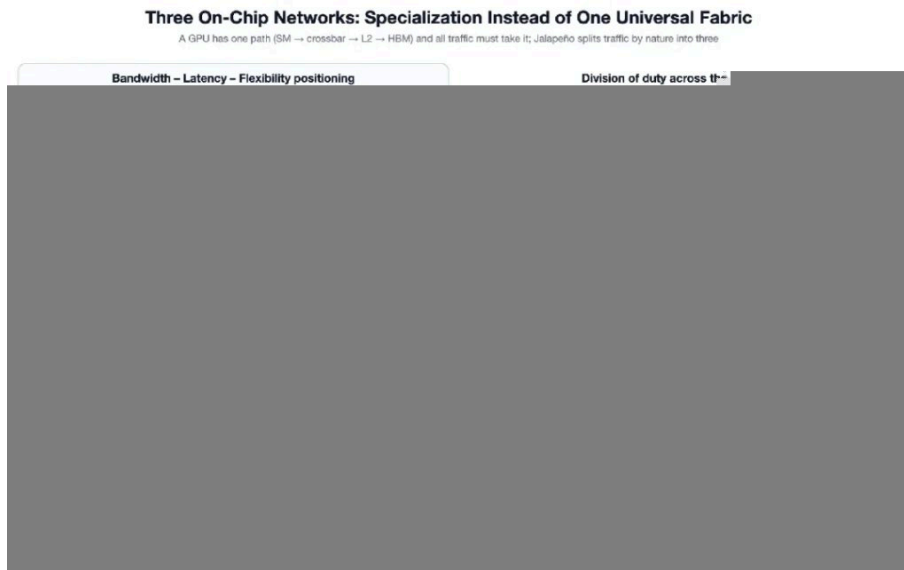
3.3 内存子系统

内存子系统最大的变化就是 NUMA 结构. 我们参考 HBM4 的一些数据, 单个 HBM4 stack 包含 64 个 pseudo-channel(PC) 位宽为 32bit, burst length = 8. 因此我们推算连接的结构如下

连接到单个 Core Slice 的位宽为 1536 bit, 每个 Core Slice 连接三颗 HBM, 总计带宽为 240GB/s, 我们可以看到这样的设计有一个好处, 比起 GPGPU 的统一内存访问, 这种做法可以降低大量的抖动, 并且延迟更低.

3.4 片上网络

整体来看, 每个 Core Slice 和它直接连接的 HBM 之间有一个 Local view, 它主要负责 Core 内部的一些 KVCache 切片和权重切片存储, 然后针对多个 Core 之间的 NUMA 结构, 整个 NOC 分为了两块, 一部分专门用于集合通信, 另一部分就是一个通用的NOC.



和GPU的最大区别就在此, GPU的编程模型决定了任意线程可以任意访问全局内存的地址, 这是CUDA生态决定的, 但是不同的内存访问速度差距非常大, 这些差距需要 L2 来填平, 然而 L2 由于物理约束, 在 Blackwell/Rubin 这样 Dual-Die 的架构中已经被分为 4 片, 它自身也存在很大的延迟差, 这样整个系统就被很大的访问延迟拖慢了. 而 Jalapeño 把这些访问内存距离的复杂性通过 NUMA 架构分离, 芯片更加简单, 而对于算子编写的复杂度交给了编译器, 而整个编译器在 LLM-Agent 的帮助下又可以进行极致的性能优化. 因此最终在芯片上, Jalapeño 获得了面积/功耗/延迟的三重收益.

我们推测 Genral NOC 是一个小带宽局部 Mesh + Global Switch 的架构, 而独特的是 Collective Network, 这里我们稍微展开分析一下, 考虑到延迟等约束, 我们仿真了几种拓扑结构, 最终认为它是一个 8x8 的 2-stage 架构, 每 8 个 Core Slice 一组, 然后全局在针对 8 组进行归并.

它需要支持 64 个 Core Slice 的 all-reduce 和 all-gather / broadcast 的能力, 并且直接连接到每个 Core Slice 的 L1. 整体上配合 Core Slice 超标量执行的结构, 它可以构建非常细粒度的 Tile based collective, 因此整体的延迟可以在 GEMM 计算时和其它算子 Fuse 的时候很好的被掩藏.

3.5 系统架构

整个系统和阿里的磐久AL128超节点类似的采用了 CPU-Tray和 Compute-Tray分离的架构, OpenAI 的CPU Tray(codename: Katsu) 为 2x Turin X86 CPU 1.5TB 内存, Compute Tray(codename: Vindaloo)包含 8 颗 Jalapeño ASIC, 两者之间通过 PCIe DAC 互联. 然后它采用了 2 套 ScaleUP 进行互联, 都使用了标准的 Ethernet 方案, 其中 Local Domain ScaleUP 支持128卡, 带宽为 600GB/s 主要用于 TP 并行, 而 Global domain 支持 2048 卡, 带宽为 200GB/s

Local Domain 采用背部的 Cable-Tray 用铜缆和 Switch-Tray(codename: Chana)互联, 而 Global Domain 则采用光互联. Switch-Tray 采用成熟的 BRCM Tomhawk 6 102.4T 以太网交换芯片. 它选择标准以太网, 相对于 Nvidia GPU 1.8TB/s ScaleUP带宽小了很多. 但是交换机直接买 Broadcom Tomahawk 6 标准件, 不需要自研 switch ASIC, 对一个 9 个月要流片的项目, 这省掉的不只是设计工作量, 更是一整条风险, 供应链与生态风险大幅降低. 这也是我在 2020 年就开始做 Ethernet ScaleUP(NetDAM)的出发点.

整个机柜架构如下右图所示:

左侧放置的是 CPU 服务器, 采用 2U 结构共计 16台, 中间的跨机柜的线缆就是PCIe DAC Cable , 每个 CPU 服务器累计 8 根, 估计采用的是 PCIe Gen5x8 的结构连接到 Compute-Tray, 或者在 Compute-Tray 内部有两颗 PCIe Switch 芯片, CPU 和 PCIe Switch 通过 4x PCIe-Gen5x16 连接. 右边上下各放置 8 个 Compute-Tray, 中间则是 6个用于 Local Domain 的1U Switch-Tray, 每个 Switch Tray 上放置 1 颗 TH6, 而用于 Global Domain 的采用的是 2U 的机框, SemiAnalysis 认为这里是包含 2 颗 TH6 的交换芯片, 但是如果是我来设计, 可能这个地方主要是放置一些 Shuffle box 或者一些小的 OCS / retimer 以及给光模块供电散热的组件即可, 而将 Global Domain 的交换机单独放置到一个机柜, 因为它是一个 Rail-based 拓扑.

整个设计的考虑是很充分的, 对于 TP 在 Local Domain 处理, 对于EP 使用更低带宽的 Global Domain. 并且对于 Jalapeño 而言, 并不需要像 Nvidia 那样攒够很大的 Batch 做 EP, 因此同步代价相对较少.

4. 软件架构

传统编译器的核心难点是在没有真实测量的情况下预测性能, 于是必须构造 cost model. Cost model 的精度决定了编译器的上限, 而在一个有 cache、有乱序、有片上网络竞争的机器上, cost model 的精度天花板很低. Jalapeño 的软件栈放弃了预测. 它把 mapping 决策变成一个可测量的搜索问题: 生成候选 → 在 chilisim 或真硅上跑 → 拿真实数字 → 迭代.

Architecture model: Simple for humans to understand

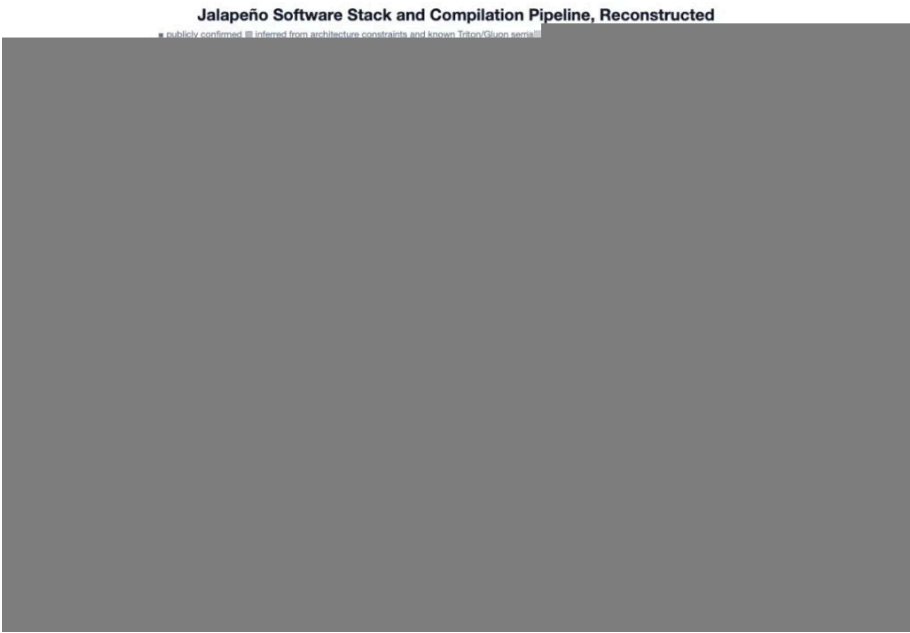


这是它和传统的 GPU 芯片最大的区别, 充分借助了现代 LLM-Agent 的能力来构建整个软件系统. 也就是 OpenAI 强调的一句话: *Spatial programming is onerous for humans. It is easy for frontier AI.*

整个系统的结构分为两层, Lowering 采用 Gluon 并 配合 Linear Layouts代数(详细分析可以参考 《学习一下 Linear Layout》). 然后 mapping 到硬件更多的使用了 LLM-Agent 进行搜索和调优.

然后整个公开的软件协议栈分为六层:

整个框架的详细过程总结如下:



这张图把 Jalapeño 的软件栈分层与本文推演的编译 Pass 流水线画在一起, 图例要先看: 实心方块是已公开确认, 斜线纹是由架构约束与 Triton/Gluon 已知语义推断, 紫色是由 AI 搜索而非 cost model 决定.

左侧分层 (Teacup / gigakernel / Gluon / Linear Layouts + TensorInfo / ~3000 行类汇编 kernel / chilisim) 每层都给了 CUDA 栈的近似对应 (vLLM ↔ Teacup, CUDA Graph ↔ gigakernel, CuTe ↔ Linear Layouts, PTX/SASS ↔ 类汇编, NCU ↔ chilisim), 但关键差异只有一条: CUDA

栈里由硬件调度器决定 CTA 落到哪个 SM, 这里由编译期 / AI 决定 program 落到哪个物理 core , 即静态空间映射取代动态调度.

右侧 P0-P7 的 Pass 流里, 标 ★ 的两个 (P2 slice 放置 / 亲和性分配, P4 tiling / 流水化 / 预取距离) 在 GPU 编译器里根本不存在: P2 决定每个权重分片、每条 KV、每个 expert 归属哪个 HBM slice , TensorInfo 就是承载这个决策的载体, 它同时编码逻辑 layout 与物理 placement; P4 则因为“硬件 L1 + 预取”没有可靠的解析 cost model (最优预取距离随 shape / 命中率 / OoO 窗口占用而变) 而天生适合交给搜索.

下方那条 DeepSeek MLA 的 40 小时实测曲线是全章最硬的证据: 功能正确 0.31% → FP8 矩阵 31.7% → 分块前瞻重缩放 59.2% → V-matmul 调度 77.1% → K-tile 预取 + 合并访存 88.9%, 逐步依次对应传统编译器的指令选择 → 数值算法重排 → 指令调度 → 访存优化 — AI 干的是 auto-scheduler + auto-tuner 的活.

图末推论回答了“为什么可以放弃通用编译器”: 硬件只需保证上界足够高 + 语义足够清晰可搜索, 找到达到上界的那条路径就从“编译器研究问题”变成“搜索算力问题”.

这也是我为什么前段时间在尝试做 Anthropic 的那个 VLIW+SIMD 的面试题, 当然不是为了找工作, 而是为了更好的深入理解整个优化流程.

我也针对 Gluon 的代码使用一些 AI 工具进行详细的分析, 具体的分析内容就不公开了..下图是一个大概的流程分析

另一方面我们需要关注 Gluon 和 XLS 的配合, 这是软硬件协同设计的很好的参考, 下一章我们将详细展开分析

5. 芯片设计

整个项目从第一行RTL开始到流片只用了 9 个月, 基本上完全颠覆了现有的芯片开发流程, 这也是我最近很长一段时间非常关注的方向.

5.1 关键前提: 先换语言, 再让 AI 上场

整个设计的关键前提是, 先换语言, 再让 AI 上场

注意这一页的版式本身就是论点: AI + DESIGNER 与 Internal AI model、Fast tooling 三者都用箭头指向中间那个蓝色的 XLS HW language , 也就是说, 在 OpenAI 自己的叙述里, XLS 是 AI 与设计者共同作用的那个基座. 我们注意到 XLS对自己的定位是:

XLS is a 'Mid-Level' Synthesis toolchain for hardware development. It is similar and has some of the same goals as High-level Synthesis (HLS) tools however it operates at a generally lower level.

四条的共同点是: 让每一步都机器可判定. 这是把“设计”从一个需要人类判断的过程, 改造成一个可以自动评分的搜索问题. 推测整个过程如下:

5.2 设计如何闭环

OpenAI 在 Hotchips 的演讲资料如下:

其中最关键的是人机的分工, 人类负责定义一些目标函数, 和选择整个芯片的主体结构, 判断哪些架构复杂是否可以删掉, 并规划整个软硬件协同的宏观架构. 而 AI 负责在结构里进行详细的高频度的穷尽搜索, 把工程师从手工调整面积/时序的工作里解放出来. 分工的边界是 *让 AI 负责擅长的在明确判据的情况下进行大规模搜索*, 而不是在一些宏观的系统架构上进行没有明确目标评判的生成代码.

6. 未来展望

6.1 对比 Nvidia GPGPU架构

我们把它和 Nvidia GPGPU 做一个完整的对比:

GPU 的存储层次是 寄存器 → L1/SMEM → (Cluster/DSMEM) → 统一 L2 (50-126 MB) → HBM, 其中统一 L2 是全芯片共享的, 所有 SM 通过全局 crossbar 访问所有 HBM 通道.

另外非常值得对比的是它们的 Tensor Unit的架构

详细做了一个对比:

从芯片面积上来看, Jalapeño 使用了更多的 tensor unit, 估计面积在 180 mm² 占整个 Die 面积的 22%, 而没有使用很大的 L2 Cache 和复杂的 RF/SMEM/TMEM/L1Cache 的内存层次结构.

Jalapeño 把 core 和 HBM 切成 **64 个 slice**, 每个 core slice 对自己那份 HBM slice 有一个 **low-latency local view**; slice 之间的同步走**专用的高带宽集合通信网络**; 另有一张**通用 NoC** 负责杂活和通往 scale-up 网络. 官方把这个称为 “minimal memory hierarchy”, 并明确说相对 GPU 的复杂内存系统这是巨大优势.

另外整个系统设计代表了两种不同的思路, Nvidia GPGPU 通过更高的 Occupancy 和 Latency hiding 获得更高的吞吐, 对于训练场景是非常有效的, 但是对于推理小batch 并且有严格SLA需求的场景, 这种方式就不适合了. 而 Jalapeño 用“缩短距离 + 预取 + 乱序窗口”直接消除延迟, 不依赖并发度.

另一方面, Transformer 每一层里, attention 输出要跨 head 合并、MLP 输出要跨 hidden 维求和. 一旦这些维度被切到多个执行单元上 (片内 slice 或片间 TP), 每层就至少需要一次归约. 层数是模型结构给定的, 归约次数因此也是给定的:

第一项随并行度线性下降, 第二项不随并行度下降, 通常还随并行度上升, 因为要跨更远的域. 因此, 在 NOC 设计上也要考虑到这方面的影响, 如果能够显著的降低这方面的延迟开销, 那么我们就可以更有效的提高 TP 并行的规模.

两种架构的对比如下:

6.2 OpenAI roadmap

在 LLM-Agent 的协作下, OpenAI宣称整个芯片(A0)回片后经过进一步优化, B0估计还有25% 性能提升已经送出流片, 未来第二代将支持训练

6.3 一些分析总结

总体来看, 在今年早些时候我就注意到了 LLM-Agent 和芯片协同设计这个巨大的机会, 从使用 LLM-Agent 详细逆向分析 Blackwell 的架构, 到自己动手把 Anthropic VLIW+SIMD challenges 做到第三名, 基本上已经完全掌握了整个迭代流程, 几个月前芯片的架构基本上也整理清楚了, 基本判断和 OpenAI 是一致的, 这也正是去年我在分析 Blackwell 架构时, 老黄在公司内部转发的那篇文章中所提到的, 一定要把现有的 warp 调度机制改变才能在 Perf-Watt Ratio 上获得数倍的提升.

而对于互联, 本来就是我最擅长的, 从 2020 年开始全球领先的 Ethernet Based ScaleUP 实现, 到现在 DPU/NIC 领域全面超越 Nvidia, 下一代在 PPA 上 DPU/NIC 也会对Nvidia 有碾压的优势, 就像 Jalapeño 这样数倍到数十倍的提升. 届时无论是 ScaleUP/ScaleOut 能和我们掰掰手腕的也没几家, 而现在算力芯片基本也准备好怎么弄了...

本文来源: [zartbot](#)

风险提示及免责条款

市场有风险, 投资需谨慎。本文不构成个人投资建议, 也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资, 责任自负。

 79  收藏

分享到:  

写评论

请发表您的评论

 表情  图片

发表评论